Cryptography
An Introduction

Continued…



Shon Harris
CISSP, 5th Edition

3.8 Types Of Symmetric Systems

- Several types of symmetric algorithms are used today. They have different methods of providing encryption and decryption functionality
- The one thing they all have in common is that they are symmetric algorithms, meaning the sender and receiver are using two instances of the same key
- In this section, we will be walking through symmetric algorithms and their characteristics

Symmetric Algorithms

- Data Encryption Standard (DES)
- 3DES (Triple DES)
- Blowfish
- Twofish
- IDEA (International Data Encryption Algorithm)
- RC4, RC5, RC6
- AES (Advanced Encryption Standard)
- SAFER (Secure and Fast Encryption Routine)
- Serpent

3.8.1 DES – Data Encryption Standard

- NSA announced in 1986 that, as of January 1988, the agency would no longer endorse DES and that DES-based products would no longer fall under compliance with Federal Standard 1027.
- The NSA felt that because DES had been so popular for so long, it would surely be targeted for penetration and become useless as an official standard.

In 1998, the Electronic Frontier Foundation built a computer system for $250,000 that broke DES in three days by using a brute force attack against the keyspace

It contained 1,536 microprocessors running at 40MHz, which performed 60 million test decryptions per second per chip

Although most people do not have these types of systems to conduct such attacks, as Moore's Law holds true and microprocessors increase in processing power, this type of attack will become more feasible for the average attacker

This brought about 3DES, which provides stronger protection, as discussed later in the chapter

DES was later replaced by the Rijndael algorithm as the ***Advanced Encryption Standard (AES)*** by NIST**.** This means that Rijndael is the new approved method of encrypting sensitive but unclassified information for the U.S. government; it has been accepted by, and is widely used in, the public arena today.

How Does DES Work ?

- DES is a symmetric block encryption algorithm. When 64-bit blocks of plaintext go in, 64-bit blocks of ciphertext come out.
- It is also a symmetric algorithm, meaning the same key is used for encryption and decryption.
- It uses a 64-bit key: 56 bits make up the true key, and 8 bits are used for parity.

How Does DES Work ? ….

- When the DES algorithm is applied to data, it divides the message into blocks and operates on them one at a time.
- The blocks are put through 16 rounds of transposition and substitution functions.
- The order and type of transposition and substitution functions depend on the value of the key used with the algorithm.
- The result is 64-bit blocks of ciphertext.

What Does It Mean When an Algorithm Is Broken?

- In most instances, an algorithm is broken if someone is able to uncover a key that was used during an encryption process.

So let's say Ali encrypted a message and sent it to Bilal. Zaheer captures this encrypted message and carries out a brute force attack on it, which means he tries to decrypt the message with different keys until he uncovers the right one.

Once he identifies this key, the algorithm is considered broken. So does that mean the algorithm is worthless?

If an algorithm is broken through a brute force attack, this just means the attacker identified the one key that was used for one instance of encryption.

But in proper implementations, we should be encrypting data with session keys, which are good only for that one session. So even if the attacker uncovers one session key, it may be useless to the attacker, in which case he now has to work to identify a new session key.

So breaking an algorithm can take place through brute force attacks or by identifying weaknesses in the algorithm itself.

Brute force attacks have increased in potency because of the increased processing capacity of computers today.

An algorithm that uses a 40-bit key has around 1 trillion possible key values. If a 56-bit key is used, then there are approximately 72 quadrillion different key values. Relative to today's computing power, these key sizes do not provide much protection at all.

DES Modes

- Block ciphers have several modes of operation. Each mode specifies how a block cipher will operate.
- One mode may work better in one type of environment for specific functionality, whereas another mode may work better in another environment with totally different requirements.
- It is important that vendors who employ DES (or any block cipher) understand the different modes and which one to use for which purpose.

DES Modes…

We will look at five DES Modes:

- Electronic Code Book (ECB)
- Cipher Block Chaining (CBC)
- Cipher Feedback (CFB)
- Output Feedback (OFB)
- Counter Mode (CTR)

Electronic Code Book Mode

- ECB mode operates like a code book. A 64-bit data block is entered into the algorithm with a key, and a block of ciphertext is produced
- For a given block of plaintext and a given key, the same block of ciphertext is always produced
- Not all messages end up in neat and tidy 64-bit blocks, so ECB incorporates padding to address this problem
- ECB is the easiest and fastest mode to use, but as we will see, it has its dangers

Electronic Code Book Mode…

- A key is basically instructions for the use of a code book that dictates how a block of text will be encrypted and decrypted.
- The code book provides the recipe of substitutions and permutations that will be performed on the block of plaintext.
- The security issue that comes up with using ECB mode is that each block will be encrypted with the exact same key, and thus the exact same code book.

Electronic Code Book Mode…

- So, two bad things can happen here:
  - An attacker could uncover the key and thus have the key to decrypt all the blocks of data, or
  - An attacker could gather the ciphertext and plaintext of each block and build the code book that was used, without needing the key.

- The crux of the problem is that there is not enough randomness to the process of encrypting the independent blocks, so if this mode is used to encrypt a large amount of data, it could be cracked more easily than the other modes that block ciphers can work in

So the next question to ask is, why even use this mode? This mode is the fastest and easiest, so we use it to encrypt small amounts of data, such as PINs, challenge-response values in authentication processes, and encrypting keys.

Because this mode works with blocks of data independently, data within a file does not have to be encrypted in a certain order.

This is very helpful when using encryption in databases. A database has different pieces of data accessed in a random fashion. If it is encrypted in ECB mode, then any record or table can be added, encrypted, deleted, or decrypted independently of any other table or record.

Other DES modes are dependent upon the text encrypted before them. This dependency makes it harder to encrypt and decrypt smaller amounts of text, because the previous encrypted text would need to be decrypted first.

Because ECB mode does not use chaining, you should not use it to encrypt large amounts of data, because patterns would eventually show themselves.

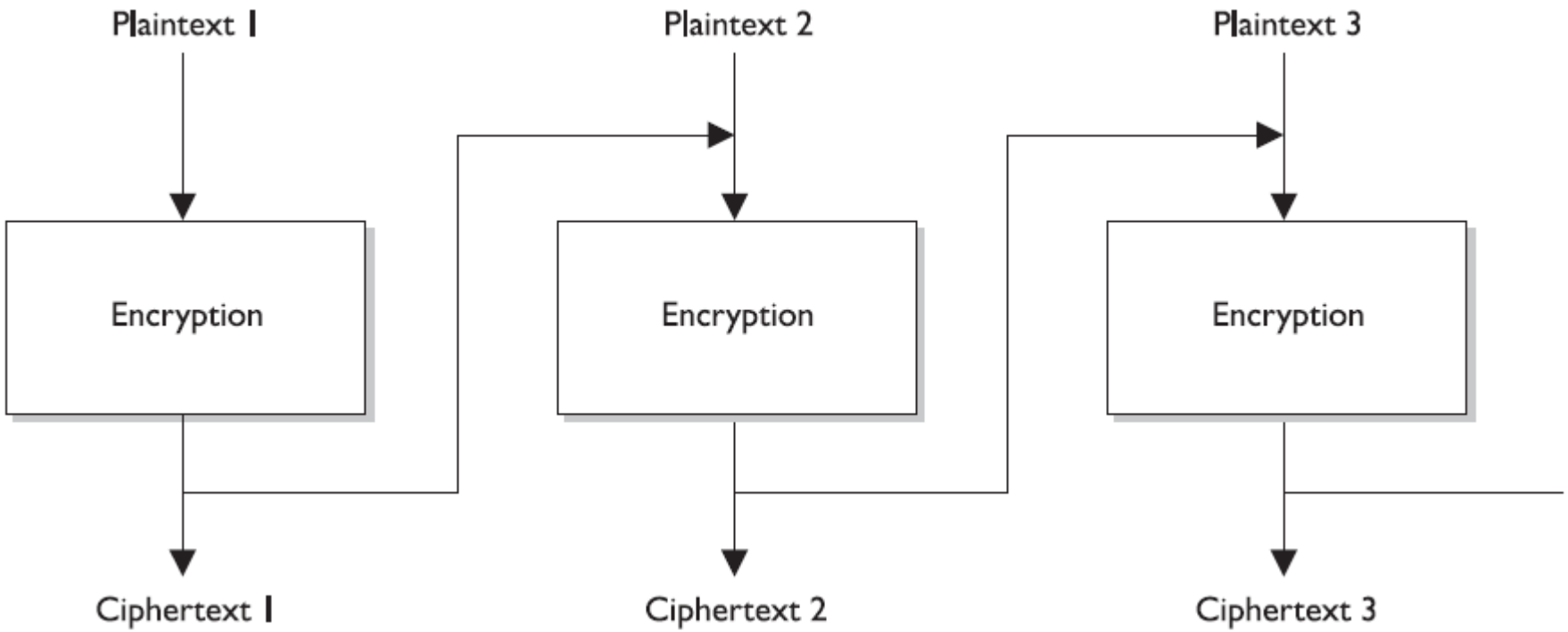
Cipher Block Chaining Mode

- In ECB mode, a block of plaintext and a key will always give the same ciphertext. This means that if the word “balloon” were encrypted and the resulting ciphertext were “hwicssn,” each time it was encrypted using the same key, the same ciphertext would always be given.

This can show evidence of a pattern, enabling an evildoer, with some effort, to discover the pattern and get a step closer to compromising the encryption process

Cipher Block Chaining Mode

- ***Cipher Block Chaining (CBC) does not reveal a pattern,*** because each block of text, the key, and the value based on the previous block are processed in the algorithm and applied to the next block of text, as shown in the Figure
- This results in more random ciphertext
- Ciphertext is extracted and used from the previous block of text. This provides dependence between the blocks, in a sense chaining them together



In CBC mode, the ciphertext from the previous block of data is used in encrypting the next block of data

This is where the name Cipher Block Chaining comes from, and it is this chaining effect that hides any patterns

The results of one block are XORed with the next block before it is encrypted, meaning each block is used to modify the following block. This chaining effect means that a particular ciphertext block is dependent upon all blocks before it, not just the previous block.

As an analogy, let's say you have five buckets of marbles. Each bucket contains a specific color of marbles: red, blue, yellow, black, and green. The first bucket of red marbles (block of bits) you shake and tumble around (encrypt) to get them all mixed up.
Then you take the second bucket of marbles, which are blue, and pour in the red marbles and go through the same exercise of shaking and tumbling them. You pour this bucket of marbles into your next bucket and shake them all up.

Cipher Block Chaining Mode

- When we encrypt our very first block using CBC, we do not have a previous block of ciphertext to “dump in” and use to add the necessary randomness to the encryption process
- If we do not add a piece of randomness when encrypting this first block, then the bad guys could identify patterns, work backward, and uncover the key.
- So, we use an initialization vector

Cipher Block Chaining Mode

- The 64-bit IV is XORed with the first block of plaintext, and then it goes through its encryption process
- The result of that (ciphertext) is XORed with the second block of plaintext, and then the second block is encrypted
- This continues for the whole message. It is the chaining that adds the necessary randomness that allows us to use CBC mode to encrypt large files

Cipher Block Chaining Mode

- Neither the individual blocks nor the whole message will show patterns that will allow an attacker to reverse-engineer and uncover the key
- If we choose a different IV each time we encrypt a message, even if it is the same message, the ciphertext will always be unique

This means that if you send the same message out to 50 people and encrypt each one using a different IV, the ciphertext for each message will be different